Лекция 11. Введение в автоматическую обработку последовательностей и естественного языка

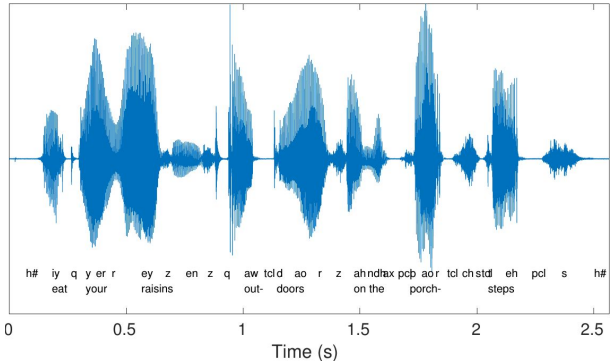
Фролов Дмитрий

Сириус, 2026

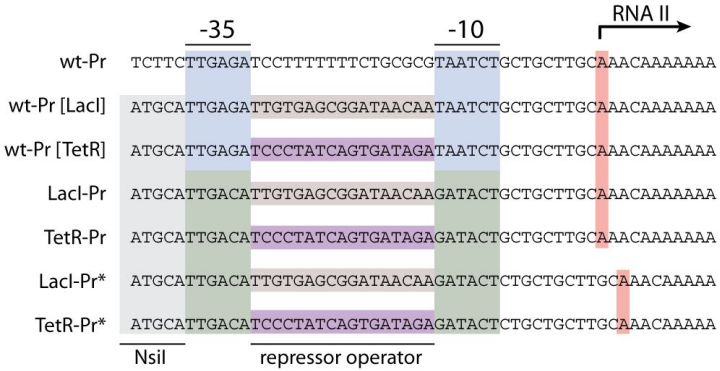
Какие бывают временные ряды?

Стоимости акций, курсы валют

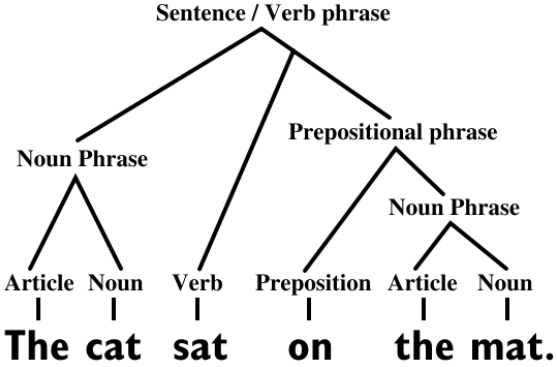
Звуки, музыка, речь



Нуклеиновые кислоты, белки

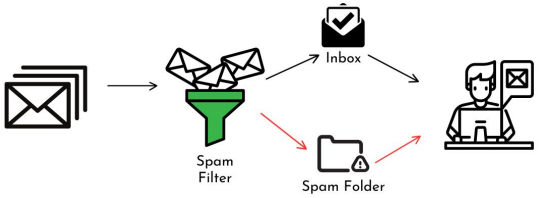


Текст

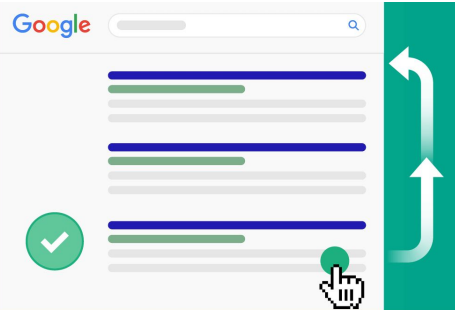


Задачи обработки текста

Классификация писем
(для фильтрации спама)



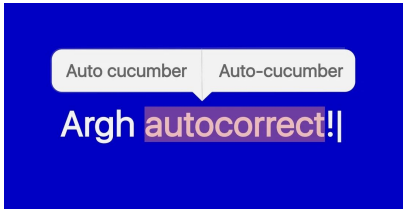
Ранжирование документов
(для поиска)



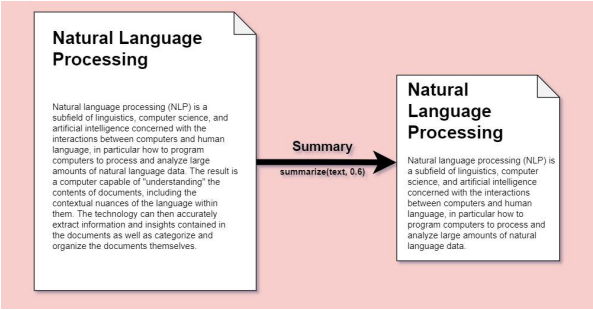
Перевод

Автодополнение

Автокоррекция



Суммаризация



Классические представления текста

Bag-of-Words

Documents	Counting word frequencies
D1	the: 1, dog: 1, is: 1, happy: 1
D2	the: 2, dog: 1, makes: 1, child: 1, happy: 1
D3	the: 2, child: 1, makes: 1, dog: 1, happy: 1

Encode	child	dog	happy	is	makes	the	BoW Vector representations
D1	0	1	1	1	0	1	[0,1,1,1,0,1]
D2	1	1	1	0	1	2	[1,1,1,0,1,2]
D3	1	1	1	0	1	2	[1,1,1,0,1,2]

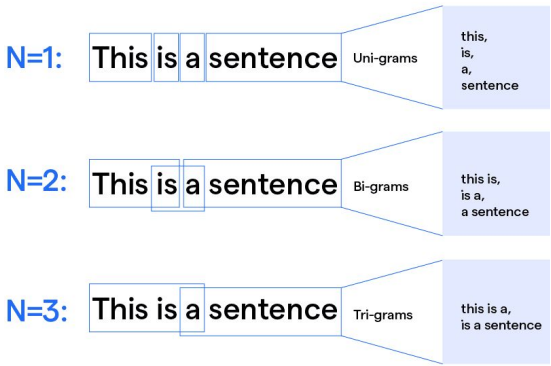
TF-IDF
(term frequency & inverse document frequency)

$$\boldsymbol{TF(t,d)} = \frac{(Number\ of\ occurrences\ of\ term\ t\ in\ document\ d)}{(Total\ number\ of\ terms\ in\ the\ document\ d)}$$

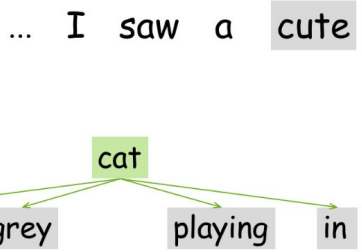
$$\boldsymbol{IDF(t,D)} = \log_e \frac{(Total\ number\ of\ documents\ in\ the\ corpus)}{(Number\ of\ documents\ with\ term\ t\ in\ them)}$$

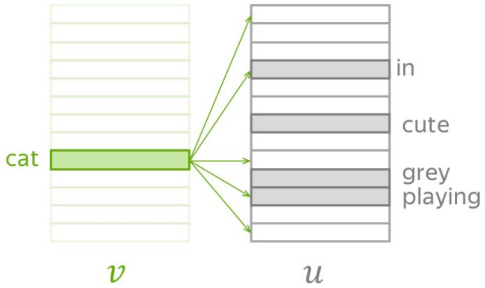
$$TF\text{-}IDF(t,d,D) = TF(t,d) \times IDF(t,D)$$

N-Grams

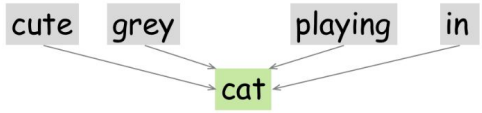


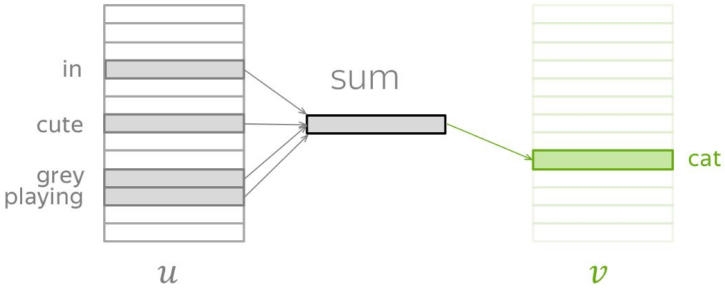
Обучаемые представления слов — *word2vec*





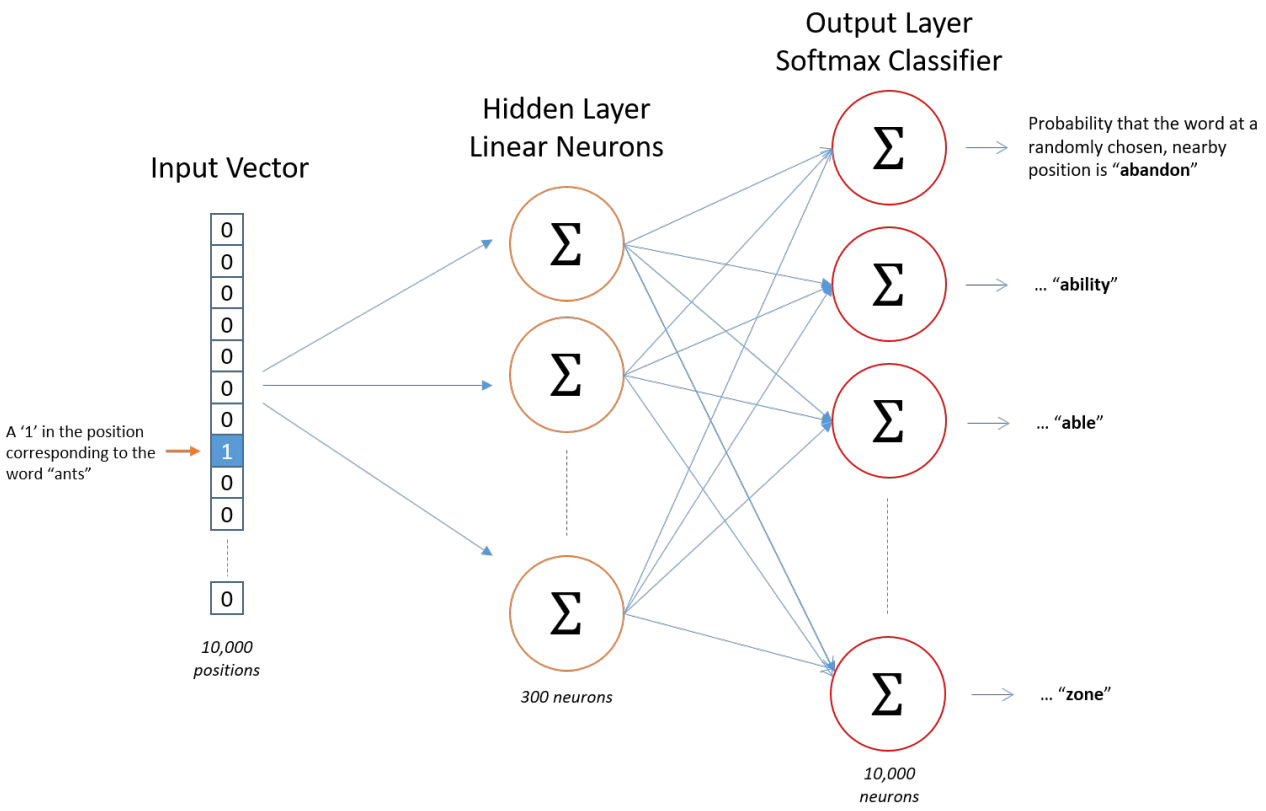
Skip-Gram: from central predict context (one at a time)



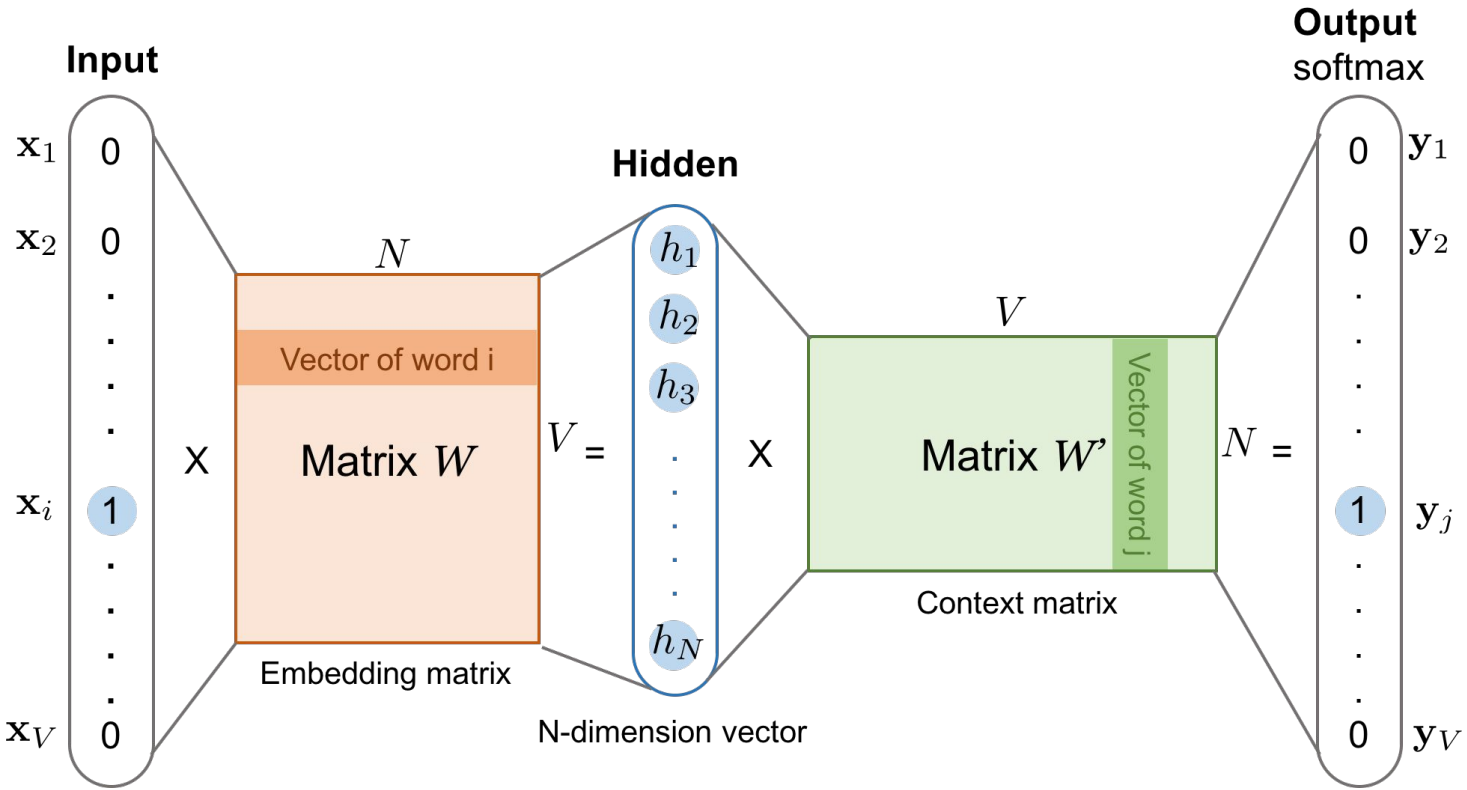


CBOW: from sum of context predict central

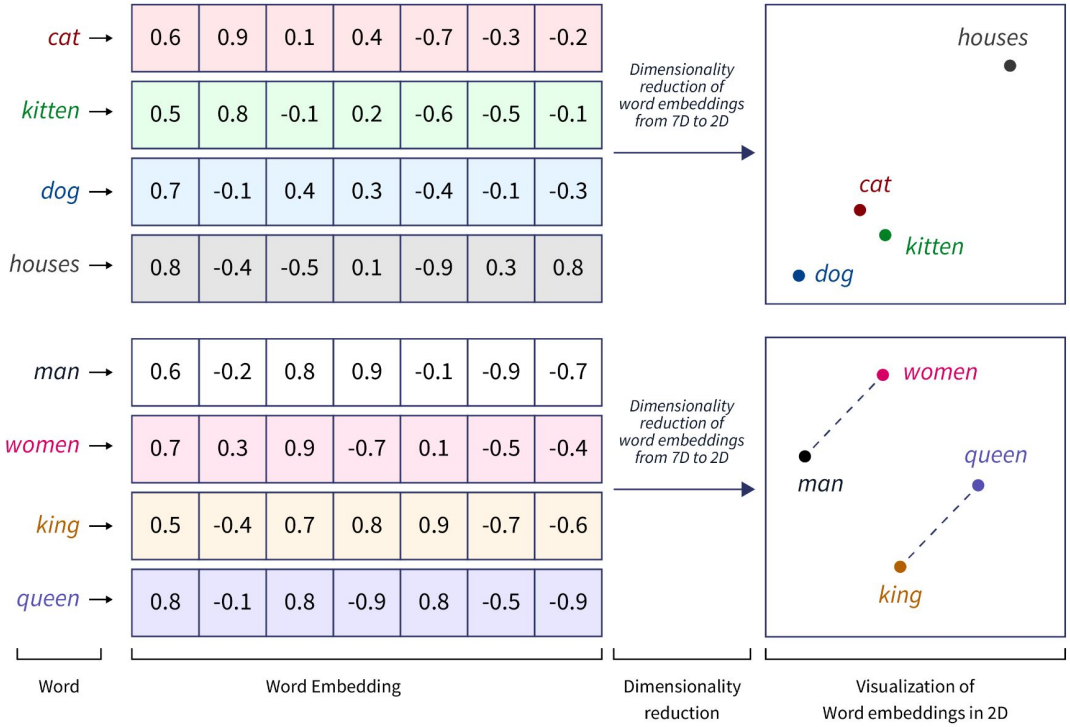
Архитектура *word2vec*



Архитектура *word2vec*



Полученные эмбеддинги имеют почти линейную связь



Виды токенизации

Let's tokenize! Isn't it easy?

1. Словарная

```
["Let's" "_" "tokenize" "!" "Isn't" "_" "it" "_" "easy" "?"]
```

2. Посимвольная

```
["L" "e" "t" "'" "s" "_" "t" "o" "k" "e" "n" "i" "z" "e" "!" "I" "s" "n" "'" "t" "_"
"i" "t" "_" "e" "a" "s" "y" "?"]
```

3. Морфемная

```
["Let" "'s" "_" "token" "ize" "!" "Is" "n't" "_" "it" "_" "easy" "?"]
```

Автоматическая токенизация (Byte-level BPE)

GPT-3.5 & GPT-4 GPT-3 (Legacy)

Many words map to one token, but some don't: indivisible.

Unicode characters like emojis may be split into many tokens containing the underlying bytes: 🖐🏾

Sequences of characters commonly found next to each other may be grouped together: 1234567890

Clear Show example

Tokens	Characters
57	252

Many words map to one token, but some don't: indivisible.

Unicode characters like emojis may be split into many tokens containing the underlying bytes: ❓❓❓❓❓❓

Sequences of characters commonly found next to each other may be grouped together: 1234567890

TEXT TOKEN IDS